

Scientific Computing

L03 Data Formats

Thorsten Koch

Applied Algorithmic Intelligence Methods department at ZIB
Chair of *Software and Algorithms for Discrete Optimization*
at TU Berlin



All emails related to this lecture should start with **SCICOMP25** in the subject, otherwise they will likely be discarded by my automatic mail sorting system.

Please send all requests regarding the course through the ISIS message system!

If you have questions about an exercise, please use the Q&A Forum.

If you need to send an email, then use the following email addresses:

Prof. Thorsten Koch koch@math.tu-berlin.de

Dr. Thai Le le@zib.de

Annika Buchholz buchholz@zib.de

Exercise submission
and grading questions

Coding was HARD Until I learned These 5 Things (Elsa Scola)

<https://www.youtube.com/watch?v=ujPDRI1q-dw>

1. Learn by doing
2. Learn to program, not a programming language.
3. Create a Roadmap. What do you want to build?
4. Prioritize understanding.
5. Get used to failing.

Topics 1: Data Formats (XML, JSON, CSV, Binary), parsing, programming by contract

1. https://en.wikipedia.org/wiki/Design_by_contract
2. Find out yourself what XML, JSON, CSV file formats are.
For XML try to figure out the difference between SAX and DOM parsers.
3. https://en.wikipedia.org/wiki/Recursive_descent_parser
4. <https://en.wikipedia.org/wiki/Unicode>
5. <https://home.unicode.org>

Related

- <https://en.wikipedia.org/wiki/XML>
- <https://en.wikipedia.org/wiki/JSON>
- CSV: <https://datatracker.ietf.org/doc/html/rfc4180>
- Binary: <https://en.wikipedia.org/wiki/Endianness>
- Parsing: <https://en.wikipedia.org/wiki/Parsing>
- Parsing: https://en.wikipedia.org/wiki/Extended_Backus%E2%80%93Naur_form

Topics 2:

- Hougardy and Vygen, Chapter 2 and 3
- [TRPL Chap 4](#) Understanding Ownership
- Look into [TRLP Chap 12](#) An I/O Project: Building a Command Line Program

Student presentation

*“The wonderful thing about standards is that
there are so many to choose from.”*

— Andrew S. Tanenbaum or Grace Hopper

Scientific Computing: Data Formats	Thorsten Koch	TU Berlin / Zuse Institute Berlin (ZIB)	12
------------------------------------	---------------	---	----

```
00000000011111111112222222222333333333344444444455555555566666666677777777778
1234567890123456789012345678901234567890123456789012345678901234567890
```

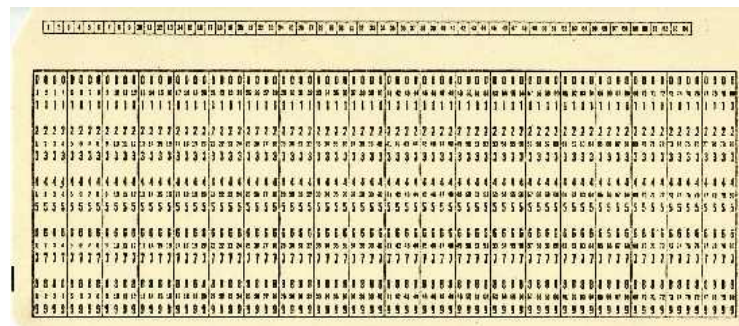
NAME	TESTPROB			
ROWS				
N	COST			
L	LIM1			
G	LIM2			
E	MYEQN			
COLUMNS				
XONE	COST	1	LIM1	1
XONE	LIM2	1		
YTWO	COST	4	LIM1	1
YTWO	MYEQN	-1		
ZTHREE	COST	3.141592653589	LIM2	1
ZTHREE	MYEQN	1		
RHS				
RHS1	LIM1	5	LIM2	10
RHS1	COST	7		
ENDATA				

Fixed width fields:
names can contain
spaces

Fixed width fields:
numbers are
truncated

RHS for Objective
can encode offset

CPLEX reads a variable
length version of this
format. But might not
give the same results.



value separator value separator ... e.g. 17,22,58,Hallo,-7,x

A CSV file is a delimited text file that uses a comma to separate values. Each line of the file is a data record. Each record consists of one or more fields, separated by commas. The use of the comma as a field separator is the source of the name for this file format. A CSV file typically stores tabular data (numbers and text) in plain text, in which case each line will have the same number of fields.

Actually, which character is used as delimiter can be chosen arbitrarily. A space or tab might do as well.

Typical problem is how to “escape” the delimiter and to ensure the delimiter is not part of a value:

- ▷ German Excel writes numbers a 17,6252. Then having “,” as separator does not work well.
- ▷ If strings are allowed as field values, then having a space as separator does not allow string with a space inside.

Main issue: You have possibly no idea what all those values mean.

There might be comment indicators in the file. Then again there is the “escape” question.

a graph

67 12

1 2 17

1 4 22

Some special character separates the data fields, often space/tab, or “;” etc.

Sometimes, special line type marker, like ‘C’ or ‘#’.

Disadvantages:

- ▷ Nearly no meta data in the file available. Do the count start at 0 ?
- ▷ Quite easy for corruption/errors to go unnoticed.
- ▷ If writing out with Excel, German version will write 0,253 instead of 0.123. (xfig did also. Couldn’t read its own files if locale was set to DE.)
- ▷ If text content, encoding unclear, escape of separation character unclear.

Advantages:

- ▷ Easy to write, usually easy to read.
- ▷ Easy to manipulate by hand.
- ▷ All classic Unix tools can manipulate: sed, awk, grep, ...

```
<?xml version="1.0" encoding="UTF-8"?>
  <DocumentElement param="value">
    <FirstElement>
      &#xb6; Some Text
    </FirstElement>
    <?some_pi some_attr="some_value"?>
    <SecondElement param2="something">
      Pre-Text <Inline>Inlined text</Inline> Post-text.
    </SecondElement>
  </DocumentElement>
```

UTF-8 Multibyte characters, but on average same size as ASCII text

UTF-16 Multiword characters, on average twice the size of ASCII text

UTF-32 Single word characters, on average 4 x the size of ASCII text.

DOM Document Object Model -> You need space to store the whole file

SAX Simple API for XML -> Faster, not much memory needed, but cannot validate

```
<?xml version="1.0" encoding="UTF-8" ?>
  <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"></xs:schema>
```

- ▷ SGML descended from IBM's Generalized Markup Language (GML), which Charles Goldfarb, Edward Mosher, and Raymond Lorie developed in the 1960s.
- ▷ The Standard Generalized Markup Language (SGML; ISO 8879:1986) is a standard for defining generalized markup languages for documents.
- ▷ The W3C XML (Extensible Markup Language) is a subset of SGML designed to ease the implementation of the parser compared to a full SGML parser, primarily for use on WWW. In addition to disabling many SGML options present in the reference syntax XML adds a number of additional restrictions on the kinds of SGML syntax.
- ▷ **XML encodes a tree structure**
- ▷ XML and its extensions have regularly been criticized for verbosity, complexity and redundancy.

```
<kwptStations xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns="http://gaslab.zib.de/kwpt/kwptstations">
  <kwptStation subnetID="waidhaus">
    <virtualConnections>
      <connectionWithCompressors type="combined" maxSerialUnits="2" maxParallelUnits="2"
        to="WAID_VS-R_A" from="WAID_VS-R_E" id="WAID_VS-R_E__WAID_VS-R_A">
        <maxOutPressure unit="barg" value="67.5"/>
        <compressorUnits>
          <compressorUnit id="GR.WAID.WAOG__.VG__.ME_2._____" />
          <compressorUnit id="GR.WAID.WAOG__.VG__.ME_4._____" />
        </compressorUnits>
      </connectionWithCompressors>
    </virtualConnections>
  </kwptStation>
</kwptStations>
```

Disadvantages:

Very verbose, i.e., high overhead. Often not too well designed.

Tedious to manipulate by hand. Need special tools for automatic manipulation.

Schemas a bit tricky to do. Using defaults changes meaning of data files. No dependencies between fields can be encoded in the schema. (e.g., see unit. Alternatives? Allowed to have different alternatives within one outer tag? Really needed?)

Tags encode tree like structure.

Advantages:

Quite verbose. Lots of meta data. Often easy to find out the meaning of fields.

If schema present can be formally validated.

means converting an in-memory object (typically a data structure like a record, list, dictionary) into a stream of bytes that can be stored (in a file) or transmitted (over a network) and later reconstructed (deserialized) back into the same data structure, though possibly on a different architecture.

Why not just dump the memory to disk and read it back in?

(super easy, super fast)

- ▷ Does not work for non continuous data structures, e.g., linked lists.
- ▷ For records (structs in C/Rust) the padding of the fields in the struct might depend on the compiler and the Rust compiler may even reorder the fields.
- ▷ If you transmit the data with a different byte ordering (big vs little endian) or different floating-point storage, the result will be garbage.
- ▷ Additionally: If somebody later looks at the memory dump it is absolutely unclear what this might be.
- ▷ Note: Writing out floating point numbers as text, might introduce rounding errors because not all decimal numbers can be represented.

```
struct ACG1 {  
    time1: u16, // 2  
    time2: u16, // 2  
    upper: u32, // 4  
    lower: u16, // 2  
}
```

// Is possibly put by the
//compiler as

```
struct ACG1 {  
    upper: u32, // 4  
    time1: u16, // 2  
    time2: u16, // 2  
    lower: u16, // 2  
    padding    // 2  
}  
= 12 bytes
```

is an open standard file format and data interchange format that uses human-readable text to store and transmit data objects consisting of attribute–value pairs and arrays (or other serializable values).

JSON exchange must be encoded in UTF-8

```
{ "firstName": "John", "lastName": "Smith", "isAlive": true, "age": 27,
  "address": {
    "streetAddress": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postalCode": "10021-3100"
  },
  "phoneNumbers": [
    { "type": "home", "number": "212 555-1234"},
    { "type": "office", "number": "646 555-4567"}
  ],
  "children": [ "Catherine", "Thomas", "Trevor" ], "spouse": null }
```

JSON is promoted as a low-overhead alternative to XML as both of these formats have widespread support for creation, reading, and decoding in the real-world situations where they are commonly used. Examples could also include CSV and YAML (a superset of JSON).

```
{ "4194304": {"authors": "Asakura, Fumioki", "issn": "0910-2043", "journal_name": "Japan J. Appl. Math 7, No. 3, 399-421 (1990).", "title": "Existence of global solutions to one-dimensional nonlinear Maxwell's equations", "author_codes": {"1": {"asakura.fumioki": 0}}, "publication_year": "1990", "citations_amount": 0, "citations": [], "classification": "35L60 35Q60", "journal_id": 1053}, "2097152": {"authors": "Aristide, Tsemo", "issn": "0002-9939; 1088-6826/e", "journal_name": "Proc. Am. Math. Soc. 132, No. 12, 3697-3702 (2004).", "title": "Closed similarity Lorentzian affine manifolds", "author_codes": {"1": {"aristide.tsemo": 1}}, "publication_year": "2004", "citations_amount": 3, "citations": [6137091, 6539939, 6310165], "classification": "53C10 53C50 53A15", "journal_id": 623}, "4194306": {"authors": "Kuttler, Kenneth", "issn": "0362-546X", "journal_name": "Nonlinear Anal., Theory Methods Appl. 15, No. 7, 601-623 (1990).", "title": "Initial boundary value problems for the displacement in an isothermal, viscous gas", "author_codes": {"1": {"kuttler.kenneth-l-jun": 0}}, "publication_year": "1990", "citations_amount": 7, "citations": [4209824, 645222, 150407, 1684845, 1882847, 2003923, 12287], "classification": "35L65 35Q35 76N15 35B40 35B65", "journal_id": 8366}, [...]
```

JSON is a compact (compared to XML) data format in a “human readable” text form and serves the **purpose of data exchange between applications**.

- ▷ JSON is independent of the programming language.
- ▷ Parsers and generators exist in all common languages.
- ▷ It is recursively defined, therefore can be arbitrary deeply nested.
- ▷ Was designed to serialize data in program structures.
- ▷ Automatic conversion between JSON and XML is to a certain degree possible.
- ▷ **You cannot interpret only part of a JSON file. You need to read the whole file.**

YAML is a human-readable **data-serialization language**.

- ▶ It is commonly used for configuration files and in applications where data is being stored or transmitted.
- ▶ YAML targets many of the same communications applications as Extensible Markup Language (XML) but has a minimal syntax which intentionally differs from SGML.
- ▶ It uses both Python-style indentation to indicate nesting, and a more compact format that uses [...] for lists and {...} for maps making **YAML 1.2 a superset of JSON**.
- ▶ You surely do not want to parse this yourself

```
---
receipt:    Oz-Ware Purchase Invoice
date:       2012-08-06
customer:
  first_name: Dorothy
  family_name: Gale

items:
  - part_no: A4786
    descrip: Water Bucket (Filled)
    price: 1.47
    quantity: 4

  - part_no: E1628
    descrip: High Heeled "Ruby" Slippers
    size: 8
    price: 133.7
    quantity: 1

bill-to: <address>
  street: |
    123 Tornado Alley
    Suite 16
  city: East Centerville
  state: KS

ship-to: *id001

specialDelivery: >
  Follow the Yellow Brick
  Road to the Emerald City.
  Pay no attention to the
  man behind the curtain.
...

```

A config file format for humans.

TOML aims to be a minimal configuration file format that's easy to read due to obvious semantics. TOML is designed to map unambiguously to a hash table. TOML should be easy to parse into data structures in a wide variety of languages.

TOML's syntax primarily consists of `key = value` pairs, `[section names]`, and `#` (for comments). TOML's syntax somewhat resembles that of .INI files, but it includes a formal specification, whereas the INI file format suffers from many competing variants.

- TOML is case-sensitive.
- A TOML file must be a valid UTF-8 encoded Unicode document.
- Whitespace means tab (0x09) or space (0x20).
- Newline means LF (0x0A) or CRLF (0x0D 0x0A).

- ▷ Text format always have the disadvantage to use much more space then necessary and needed longer to be read by the computer, e.g., to store 1 byte, at least 4 bytes text “255” + separator are needed.
- ▷ The drawback of binary formats is that you must know the format explicitly. Otherwise, it is not clear what is an integer number, a text character, a floating-point number. Additionally, edianness is an issue, regarding interchange.
- ▷ Usually when you encounter huge amounts of data and/or the data is not human readable anyway, then binary data formats are preferred.

```
0000: 7B 0A 20 20 22 66 69 72 73 74 4E 61 6D 65 22 3A {. "firstName":
0010: 20 22 4A 6F 68 6E 22 2C 0A 20 20 22 6C 61 73 74 "John",. "last
0020: 4E 61 6D 65 22 3A 20 22 53 6D 69 74 68 22 2C 0A Name": "Smith",.
0030: 20 20 22 69 73 41 6C 69 76 65 22 3A 20 74 72 75 "isAlive": tru
0040: 65 2C 0A 20 20 22 61 67 65 22 3A 20 32 37 2C 0A e,. "age": 27,.
00F0: 20 20 20 20 22 74 79 70 65 22 3A 20 22 68 6F 6D "type": "hom
```

```
0000: 7F 45 4C 46 02 01 01 00 00 00 00 00 00 00 00 00  ELF.....
0010: 02 00 3E 00 01 00 00 00 F0 44 40 00 00 00 00 00  ..>.....D@.....
0020: 40 00 00 00 00 00 00 00 38 DE 1D 00 00 00 00 00  @.....8.....
```

Disadvantages:

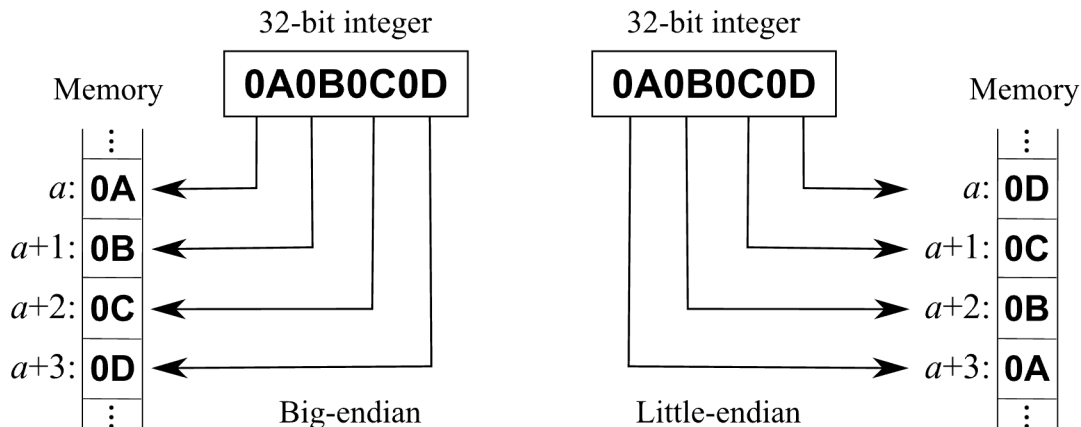
Without a specification just a useless heap of bytes.

Difficult to manipulate by hand. A few general tools available, but manipulation usually needs format specific tools.

No visual inspection.

Advantages:

Less redundant, sometimes can be read directly into data structures, often quite efficient.



By R. S. Shaw - Own work, Public Domain, <https://commons.wikimedia.org/w/index.php?curid=2974661>

TIFF: [Tag Image File Format](#), abbreviated TIFF or TIF, is an image file format for storing raster graphics images, popular among graphic artists, the publishing industry, and photographers. TIFF is widely supported by scanning, faxing, word processing, optical character recognition, image manipulation, desktop publishing, and page-layout applications.

PARQUET: [Apache Parquet](#) is a free and open-source column-oriented data storage format in the Apache Hadoop ecosystem.

MP4: [MPEG-4 Part 14 or MP4](#) is a digital multimedia container format most commonly used to store video and audio, but it can also be used to store other data such as subtitles and still images. Like most modern container formats, it allows streaming over the Internet.

PDF: [Portable Document Format](#), is a file format to present documents, including text formatting and images, in a manner independent of application software, hardware, and operating systems. Based on the PostScript language, each PDF file encapsulates a complete description of a fixed-layout flat document, including the text, fonts, vector graphics, raster images and other information needed to display it.

Lack of Turing-completeness can be a feature. Take PDF vs PostScript. Postscript is Turing-complete and therefore you cannot jump to an arbitrary page or even know how many pages the document has without running the entire thing first. By limiting expressiveness, you also gain static analysis and predictability. It's not about limiting the potential of computers; it's about designing systems that strike the right balance between the power given to the payload and the guarantees offered to the container/receiver (from <https://news.ycombinator.com/item?id=7362839>)

Now take into account that there is no difference between Data and Code in a von Neuman Architecture

- ▶ Can the file format be **streamed**, i.e., can we interpret the content without need for random access? (like, e.g., needed with many types of TIFF files, SAX vs. DOM parsing in XML.)
- ▶ There are many **CONTAINER** formats. These are just structure for some other content, e.g., `tar`
- ▶ There are many data **compression** methods, e.g., `gzip`, `xz`, `rle`, ...
It is important to distinguish between (general) lossless compression and lossy compression. The later, like, e.g., `jpeg`, `mp3`, `mp4`, just produces a look-a-like (or hear-a-like).
- ▶ Files containing source code and executable programs are also data files.
- ▶ It is a valid way to encode data in a program that reproduces it, see, e.g., `postscript`.
- ▶ Nevertheless, working with Turing complete or in general very complex data file is usually hard.
- ▶ For complex formats you might want to use a parser generator.
- ▶ **A converter between two formats is ideally a homomorphism between the two sets of possible inputs. This is rarely achievable. So be aware when converting files to other formats.**
- ▶ Apache Hadoop is an open-source framework for scalable, distributed software. It enables intensive computing processes with large amounts of data (big data, **petabyte range**) to be performed on computer clusters.

- ▶ P.138: *Real world data is dirty.*
- ▶ P.145: *Using tqdm* will occasionally make your code flaky...*
Be also aware what happen if you write your output to a log file. Really long running code will usually be executed on some server in the background, so interactive output is not desirable. (*tqdm is a python library for displaying progress bars).
- ▶ Be aware that what is put in Chapters 9 “Getting Data” and 10 “Working with Data” will usually take most of your time.
And its not really much fun.
Being able to do this efficiently, allows you to spend your time on the interesting part of data analysis.

```
fn main() {  
    let c: char = 65 as char;  
    print!("{}", c);  
  
    let b: char = 0x42 as char;  
    print!("{}", b);  
  
    let a: char = 0b1000011 as char;  
    print!("{}", a);  
}
```

ABC

7bit ASCII Chart from Wikipedia

Code	...0	...1	...2	...	...4	...5	...6	...7	...8	...9	...A	...B	...C	...D	...E	...F
0...	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1...	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	G	RS	US
2...	SP	!	"	#	\$	%	&	'	(	)	*	+	,			
3...	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4...	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5...	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6...	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7...	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL

Ctrl-C

Ctrl-D

Ctrl-G
'\a'

Ctrl-H

Ctrl-I
'\t'

Ctrl-J
'\n'

Ctrl-M
'\r'

Unix: LF, Mac: CR, DOS/Windows CR+LF



Depending how your system is setup, i.e., where you are etc. you would see different characters.

This is why Metadata is important. One not only needs the file with the text, but also the information how to interpret this text.

Otherwise, all you have is a stream of bytes. And that can as well be a picture.

437	Englisch
708	Arabisch (ASMO)
720	Arabisch (Microsoft)
737	Griechisch
775	Baltisch
850	Westeuropäisch
852	Mitteuropäisch
855	Kyryllisch
857	Türkisch
858	Westeuropäisch mit Euro
860	Portugiesisch
861	Isländisch
862	Hebräisch
863	Kanadisches Französisch
864	Arabisch (IBM)
865	Nordisch
866	Russisch
869	Griechisch

Von Drmcreeedy - Self-made using this perl script and information from Unicode.,
Gemeinfrei, <https://commons.wikimedia.org/w/index.php?curid=50132011>

00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F
20	21	22	23	24	25	26	27	28	29	2A	2B	2C	2D	2E	2F
30	31	32	33	34	35	36	37	38	39	3A	3B	3C	3D	3E	3F
40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F
60	61	62	63	64	65	66	67	68	69	6A	6B	6C	6D	6E	6F
70	71	72	73	74	75	76	77	78	79	7A	7B	7C	7D	7E	7F
80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F
90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF
B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF
C0	C1	C2	C3	C4	C5	C6	C7	C8	C9	CA	CB	CC	CD	CE	CF
D0	D1	D2	D3	D4	D5	D6	D7	D8	D9	DA	DB	DC	DD	DE	DF
E0	E1	E2	E3	E4	E5	E6	E7	E8	E9	EA	EB	EC	ED	EE	EF
F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	FA	FB	FC	FD	FE	FF

- Lateinische Schriften und Symbole
- Andere europäische Schriften
- Afrikanische Schriften
- Nahost- und Südwestasiatische Schriften
- Süd- und Mittelasiatische Schriften
- Südostasiatische Schriften
- Ostasiatische Schriften
- CJK-Ideogramme
- Indonesische und ozeanische Schriften
- Amerikanische Schriften
- Notationssysteme
- Symbole
- Privater Nutzungsbereich
- UTF-16-Surrogates

Stand: Unicode 15.0



<https://www.unicode.org/charts>

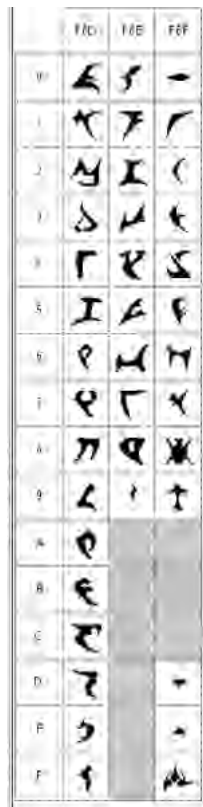
Zanabazar Square

	230	231	232	233	234	235	236	237	238	239	23A	23B	23C	23D	23E	23F
0	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	G
2	2	3	4	5	6	7	8	9	A	B	C	D	E	F	G	H
3	3	4	5	6	7	8	9	A	B	C	D	E	F	G	H	I
4	4	5	6	7	8	9	A	B	C	D	E	F	G	H	I	J
5	5	6	7	8	9	A	B	C	D	E	F	G	H	I	J	K
6	6	7	8	9	A	B	C	D	E	F	G	H	I	J	K	L
7	7	8	9	A	B	C	D	E	F	G	H	I	J	K	L	M
8	8	9	A	B	C	D	E	F	G	H	I	J	K	L	M	N
9	9	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U

The Unicode Standard, Version 15.0, Copyright © 1991-2022 Unicode, Inc. All rights reserved.

	11A0	11A1	11A2	11A3	11A4
0	0	1	2	3	4
1	1	2	3	4	5
2	2	3	4	5	6
3	3	4	5	6	7
4	4	5	6	7	8
5	5	6	7	8	9
6	6	7	8	9	A
7	7	8	9	A	B
8	8	9	A	B	C
9	9	A	B	C	D
A	A	B	C	D	E
B	B	C	D	E	F
C	C	D	E	F	G
D	D	E	F	G	H
E	E	F	G	H	I
F	F	G	H	I	J

Tengwar and Klingon still missing



	Témar (columns)			
	I (tincoléma)	II (parmatéma)	III (calmatéma)	IV (quessetéma)
Tyeller (rows)	1	p [p] tico	p [p] parma	c [k] calma qu [kʰ]
	2	nd [nd] ando	mb [mb] umbar	ng [ŋ] anga ngw [ŋʷ]
	3	b [b] thúle = súle	f [f] formen	ch [x] harmo hwesta
	4	nt [nt] anto	mp [mp] ampa	nc [ŋk] anca nqu [ŋkʰ]
	5	n [n] númen	m [m] malta	n [n] ngoldo ngwalme
	6	r [r] óre	vw [vw] vala	o [o] anna vilya
Additional tengwar	7	r [r] rómen	rd [rd] arda	l [l] lambe
	8	s [s] silme	s [s] sime nuquerna	z [z] esse = áze nuquerna
	9	hy [hy] hyarnen	hw [hw] hwesta sindarinwa	i [i] yanta ely [ely]
	10	a [a] osse	h [h] gasdil = halla	u [u] telo
	11	- []	- []	- []
	12	- []	- []	- []

Legend: t Classic Mode (for Quenya) t Mode of Beleriand (for Sindarin) t General Mode (for Sindarin, Adunaic, Black Speech, etc.) t Pronunciation (Indications in IPA)

A file is just a sequence of bytes.

The memory (RAM) of a computer is just filled with bytes.

There is no inherent distinction of program, code, data, text, ...

You can map the contents of a file into some region of memory.

What it is depends on what you do with it.

You can read a poem by Shakespeare and execute it as a program.

You can send some part of memory that contains a program in machine code to the printer.

However, neither will work very well.

How to interpret some bytes is usually decided by available meta information.

If the filename ends in .png we can assume it might be an image in PNG format. By checking the first 8 bytes we can get further assurance. If they read 0x89 0x50 0x4E 0x47 0x0D 0x0A 0x1A 0x0A it very likely is. From then we follow the structure of the format to decide whether it is a valid PNG file and decode it.

Student presentation

(DbC), is an approach for designing software.

It prescribes that software designers should define formal, precise and verifiable interface specifications for software components, which extend the ordinary definition of abstract data types with preconditions, post-conditions and invariants.

These specifications are referred to as “contracts”, in accordance with a conceptual metaphor with the conditions and obligations of business contracts.

- ▶ **Preconditions** — What must be true *before* calling the routine (e.g., problem well-posed, parameters valid). If violated, the algorithm’s guarantees don’t apply.
- ▶ **Postconditions** — What the routine guarantees when it returns (e.g., accuracy/tolerance met, structure preserved).
- ▶ **Invariants** — Properties that remain true throughout execution (often inside iterations), used to reason about correctness and convergence.

<https://www.eiffel.com/values/design-by-contract/introduction>

https://en.wikipedia.org/wiki/Design_by_contract (German Wiki page is better)

<https://crates.io/crates/contracts>

Example: Compute the square root of x, see Hugardy, Vygen, Chapter 5

```
const MAX_ERROR: f32 = 1e-6;
```

```
fn squareroot_binsearch(x: f32) -> f32 {
```

```
    let mut lower    = 1.0;  
    let mut upper    = x;  
    let mut mid      = (lower + upper) / 2.0;  
    let mut residual = (mid * mid - x).abs();
```

```
    while residual > MAX_ERROR {  
        if mid * mid < x {  
            lower = mid;  
        } else {  
            upper = mid;  
        }  
        mid      = (lower + upper) / 2.0;  
        residual = (mid * mid - x).abs();  
    }  
    mid
```

```
x=6.0
```

```
mid=2.250000 residual=0.937500 lower=1.000000 upper=3.500000  
mid=2.875000 residual=2.265625 lower=2.250000 upper=3.500000  
mid=2.562500 residual=0.566406 lower=2.250000 upper=2.875000  
mid=2.406250 residual=0.209961 lower=2.250000 upper=2.562500  
mid=2.484375 residual=0.172119 lower=2.406250 upper=2.562500  
mid=2.445312 residual=0.020447 lower=2.406250 upper=2.484375  
mid=2.464844 residual=0.075455 lower=2.445312 upper=2.484375  
mid=2.455078 residual=0.027409 lower=2.445312 upper=2.464844  
mid=2.450195 residual=0.003457 lower=2.445312 upper=2.455078  
mid=2.447754 residual=0.008501 lower=2.445312 upper=2.450195  
mid=2.448975 residual=0.002523 lower=2.447754 upper=2.450195  
mid=2.449585 residual=0.000466 lower=2.448975 upper=2.450195  
mid=2.449280 residual=0.001029 lower=2.448975 upper=2.449585  
mid=2.449432 residual=0.000281 lower=2.449280 upper=2.449585  
mid=2.449509 residual=0.000093 lower=2.449432 upper=2.449585  
mid=2.449471 residual=0.000094 lower=2.449432 upper=2.449509  
mid=2.449490 residual=0.000001 lower=2.449471 upper=2.449509  
search: 2.449490
```

What happens for x=0.5 ?


```
mid=5.250000 residual=9.562500 lower=1.000000 upper=9.500000
mid=3.125000 residual=8.234375 lower=1.000000 upper=5.250000
mid=4.187500 residual=0.464844 lower=3.125000 upper=5.250000
mid=4.718750 residual=4.266602 lower=4.187500 upper=5.250000
mid=4.453125 residual=1.830322 lower=4.187500 upper=4.718750
mid=4.320312 residual=0.665100 lower=4.187500 upper=4.453125
mid=4.253906 residual=0.095718 lower=4.187500 upper=4.320312
mid=4.220703 residual=0.185665 lower=4.187500 upper=4.253906
mid=4.237305 residual=0.045250 lower=4.220703 upper=4.253906
mid=4.245605 residual=0.025166 lower=4.237305 upper=4.253906
mid=4.241455 residual=0.010059 lower=4.237305 upper=4.245605
mid=4.243530 residual=0.007549 lower=4.241455 upper=4.245605
mid=4.242493 residual=0.001255 lower=4.241455 upper=4.243530
mid=4.243011 residual=0.003147 lower=4.242493 upper=4.243530
mid=4.242752 residual=0.000946 lower=4.242493 upper=4.243011
mid=4.242622 residual=0.000154 lower=4.242493 upper=4.242752
mid=4.242687 residual=0.000395 lower=4.242622 upper=4.242752
mid=4.242655 residual=0.000120 lower=4.242622 upper=4.242687
mid=4.242639 residual=0.000017 lower=4.242622 upper=4.242655
mid=4.242647 residual=0.000051 lower=4.242639 upper=4.242655
mid=4.242642 residual=0.000015 lower=4.242639 upper=4.242647
mid=4.242640 residual=0.000002 lower=4.242639 upper=4.242642
mid=4.242641 residual=0.000006 lower=4.242640 upper=4.242642
mid=4.242641 residual=0.000002 lower=4.242640 upper=4.242641
mid=4.242640 residual=0.000002 lower=4.242640 upper=4.242641
mid=4.242640 residual=0.000002 lower=4.242640 upper=4.242641
...
```

The program does not terminate. Why?

```
mid=4.242640 residual=0.000002 lower=4.242639 upper=4.242642
01000000100001111100001110110010 lower
01000000100001111100001110111010 upper
01000001000001111100001110110110 lower+upper
01000000100001111100001110110110 mid
mid=4.242641 residual=0.000006 lower=4.242640 upper=4.242642
01000000100001111100001110110110 lower
01000000100001111100001110111010 upper
01000001000001111100001110111000 lower+upper
01000000100001111100001110111000 mid
mid=4.242641 residual=0.000002 lower=4.242640 upper=4.242641
01000000100001111100001110110110 lower
01000000100001111100001110111000 upper
01000001000001111100001110110111 lower+upper
01000000100001111100001110110111 mid
mid=4.242640 residual=0.000002 lower=4.242640 upper=4.242641
01000000100001111100001110110110 lower
01000000100001111100001110110111 upper
01000001000001111100001110110110 lower+upper
01000000100001111100001110110110 mid
```

Because lower—upper is so small, that

Lower + (lower—upper)/2 == lower

Happens also, e.g., with 77 at f64

Contract is: if the input $x \geq 1$ then the output r will satisfy $r - x^2 \leq 10^{-6}$

```
const MAX_ERROR: f32 = 1e-6;
fn squarerooot_binsearch(x: f32) -> f32 {
    assert!(x >= 1.0); // Precondition
    let mut lower    = 1.0;
    let mut upper    = x;
    let mut mid      = (lower + upper) / 2.0;
    let mut residual = (mid * mid - x).abs();
    while residual > MAX_ERROR {
        if mid * mid < x {
            lower = mid;
        } else {
            upper = mid;
        }
        mid      = (lower + upper) / 2.0;
        assert!(residual > (mid * mid - x).abs());
        residual = (mid * mid - x).abs();
        assert!(lower * lower < mid * mid);
        assert!(mid * mid < upper * upper);
        assert!(lower * lower < x && x < upper * upper);
        assert!(mid > lower && mid < upper);
    }
    assert!((mid * mid - x).abs() <= MAX_ERROR); // Postcond.
    mid
}
```

Imagine it like a theorem+proof:

Provided $x \geq 1$

Theorem: at the end r will satisfy $r - x^2 \leq 10^{-6}$

Proof:

The following is always true:

$$l^2 < x^2 < u^2$$

$$l^2 < m^2 < u^2$$

$$r_i = |x^2 - m^2|$$

m is the middle of the interval (l, u) .

In each step the interval is halved to (l, m) or (m, u) .

Therefore: $r_i < r_{i-1}$

If the residual gets smaller in every iteration, the algorithm will converge at some point. ■

Contract is: if the input $x \geq 1$ then the output r will satisfy $r - x^2 \leq 10^{-6}$

```
fn squareroot(x: f32) -> f32 {
    assert!(x >= 1.0); // Precondition: otherwise, all bets are off.

    const MAX_ERROR: f32 = 1e-6;
    let mut result      = 0.5 * x;
    let mut residual    = (result * result - x).abs();

    while residual > MAX_ERROR {
        result      = 0.5 * (result + x / result);
        let r_new = (result * result - x).abs();

        assert!(r_new < residual); // Invariant: necessary for the loop to terminate
        residual = r_new;
    }
    assert!((result * result - x).abs() <= MAX_ERROR); // Postcondition

    result
}
```

Which of the two algorithms
converges faster?

Tokenizer, Parser, Grammar, Parser generator, RPN

LL parser (Left-to-right, leftmost derivation) is a top-down parser for a restricted context-free language. It parses the input from Left to right, performing Leftmost derivation of the sentence.

An LL parser is called an LL(k) parser if it uses k tokens of lookahead when parsing a sentence. A grammar is called an LL(k) grammar if an LL(k) parser can be constructed from it. A formal language is called an LL(k) language if it has an LL(k) grammar. The set of LL(k) languages is properly contained in that of LL($k+1$) languages, for each $k \geq 0$. A corollary of this is that not all context-free languages can be recognized by an LL(k) parser.

LR parser (Left-to-right, rightmost derivation in reverse) is a bottom-up parser. It reads input text from left to right and produces a rightmost derivation in reverse. To avoid backtracking or guessing, the LR parser is allowed to peek ahead at k lookahead input symbols before deciding how to parse earlier symbols.

LR parsers are deterministic; they produce a single correct parse without guesswork or backtracking, in linear time.

LR parsers can handle a larger range of languages and grammars than top-down LL parsing. This is because the LR parser waits until it has seen an entire instance of some grammar pattern before committing to what it has found. An LL parser must decide or guess what it is seeing much sooner, when it has only seen the leftmost input symbol of that pattern.

```
input:      /* empty string */
           | input line
;
line:       '\n'
           | exp '\n' { printf ("\t%.10g\n", $1); }
;
exp:        NUM                { $$ = $1;          }
           | exp '+' exp        { $$ = $1 + $3;    }
           | exp '-' exp        { $$ = $1 - $3;    }
           | exp '*' exp        { $$ = $1 * $3;    }
           | exp '/' exp        { $$ = $1 / $3;    }
           | '-' exp %prec NEG { $$ = -$2;        }
           | exp '^' exp        { $$ = pow ($1, $3); }
           | '(' exp ')'        { $$ = $2;        }
;
```

<https://github.com/iliabyllich/rust-bison-skeleton>

Bison is a parser generator, and in fact it doesn't really care what's your programming language.

Under the hood it takes your .y file, parses it, extracts all derivations and then constructs a bunch of tables. Then, this data is passed to a template that is called skeleton.

This skeleton is a special file written in M4macro language that (once rendered) prints your .rs file. As simple as that.

bison(1) is the GNU replacement for yacc(1) .y or .yy

flex(1) is the GNU replacement for lex(1) .l

Bison converts this grammar into C or C++ code that can parse input according to the grammar.

- ▶ Use descriptive names for globals, short names for locals
- ▶ Be consistent
- ▶ Use active names for functions
- ▶ Be accurate
- ▶ Indent to show structure
- ▶ Use the natural form for expressions
- ▶ Parenthesize to resolve ambiguity
- ▶ Break up complex expressions
- ▶ Be clear
- ▶ Be careful with side effects
- ▶ Use a consistent indentation and brace style
- ▶ Use idioms for consistency
- ▶ Give names to magic numbers

```
int f1(void) { cout << "f1 "; return 2; }
int f2(void) { cout << "f2 "; return 3; }
int f3(void) { cout << "f3 "; return 5; }
int f4(void) { cout << "f4 "; return 7; }
```

```
int ff(int a, int b, int c) {
    cout << "ff ";
    return a + b + c;
}
```

```
int main() {
    auto x = ff(f1(), f2() * f3(), f4());
    cout << "x=" << x << endl;
}
```

f4 f2 f3 f1 ff x=24

```
fn f1() -> i32 { print!("f1 "); 2 }
fn f2() -> i32 { print!("f2 "); 3 }
fn f3() -> i32 { print!("f3 "); 5 }
fn f4() -> i32 { print!("f4 "); 7 }
```

```
fn ff(a: i32, b: i32, c: i32) -> i32 {
    print!("ff ");
    a + b + c
}
```

```
fn main() {
    let x = ff(f1(), f2() * f3(), f4());
    println!("x={}", x);
}
```

f1 f2 f3 f4 ff x=24

An operation, function or expression is said to have a side effect if it modifies some state variable value(s) outside its local environment, which is to say if it has any observable effect other than its primary effect of returning a value to the invoker of the operation.

Example side effects include modifying a non-local variable, modifying a static local variable, modifying a mutable argument passed by reference, performing I/O or calling other functions with side-effects. In the presence of side effects, a program's behaviour may depend on history; that is, the order of evaluation matters. Understanding and debugging a function with side effects requires knowledge about the context and its possible histories.

Side effects play an important role in the design and analysis of programming languages. For example, imperative programming is commonly used to produce side effects, to update a system's state. By contrast, declarative programming is commonly used to report on the state of system, without side effects. Functional programming aims to minimize or eliminate side effects. *The lack of side effects makes it easier to do formal verification of a program.*

Eliminating side effects, i.e., changes in state that do not depend on the function inputs, can make it much easier to understand and predict the behavior of a program, which is one of the key motivations for the development of functional programming.

A mathematical function $f(x) \rightarrow \mathbb{N}, x \in \mathbb{N}$ is side effect free. This means, for a given x the result $f(x)$ will always be the same. Therefore, the compiler could optimize the code on the left.

Side effect means:

It makes a difference, whether you call $f(x)$ once or twice.

```
56
17
17
17
17
17
1042201148
3524153451
1973856974
1276228565
3066622768
```

```
fn f(x : u32) -> u32 {
    x * 3 + 5
}

fn main() {
    let mut x = 0;
    for _ in 0..5 {
        x = f(17);
    }
    println!("{x}");

    for _ in 0..5 {
        println!("{x}");
    }

    let mut rng = Lcg::new(17);
    for _ in 0..5 {
        x = rng.next();
    }
    println!("{x}");
}
```

```
use std::io::{self, Write};

fn f(count: usize) {
    let mut a: [usize; 2] = [0; 2];
    a[1] = count + 1;
    if count % 1000 == 0 {
        print!("{}", count / 1000);
        io::stdout().flush().unwrap(); // Flush stdout
    }
    if count < 1_000_000 { f(a[1]); }
}

fn main() { f(1); }
```

```
$ cargo run
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43
44 45 46 47 48 49 50 51 52
thread 'main' has overflowed its stack
fatal runtime error: stack overflow
Aborted (core dumped)
```

```
fn f(count: usize) {
    let mut a: [usize; 100] = [0; 100];
    a[1] = count + 1;
    if count % 1000 == 0 {
        print!("{}", count / 1000);
        io::stdout().flush().unwrap();
    }
    if count < 1_000_000 { f(a[1]); }
}
```

```
$ cargo run
1 2 3 4 5 6 7 8
thread 'main' has overflowed its stack
fatal runtime error: stack overflow
Aborted (core dumped)
```

```
$ ulimit -s 16384
$ cargo run
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
thread 'main' has overflowed its stack
fatal runtime error: stack overflow
Aborted (core dumped)
```

Fragen

有問題嗎

คำถาม

?

質問

Вопросы

Questions

Câu hỏi